Capability Routing for Agent Skill Libraries

A taxonomy, a router, and an evidence-driven learning loop for over 4,200 installed skills

Aswath A S

Independent researcher

ABSTRACT

Coding agents increasingly ship with large libraries of skills: reusable instruction bundles that each describe one kind of job. Surfacing every skill in the prompt is unaffordable, and choosing the wrong one wastes the context window and the user's time. We present a capability-routing system for agent skill libraries, built and evaluated on one machine with over 4,200 installed skills. The system has four coupled parts: a host-level catalog gate that hides skills the agent never needs, a hand-curated capability taxonomy with evidence-based tagging, a router that resolves a free-text need to exactly one skill, and a learning loop that scores skills from what actually happens in a session rather than from the request text. The router reaches 0.849 capability top-1 accuracy against 0.614 for a faithfully reimplemented pre-fix matcher and 0.473 for Okapi BM25 over skill descriptions, and raises an on-topic rate from 0.460 to 0.799, while the runtime catalog falls from 1,387 skills to 44 and the catalog listing cost from 1.92M tokens to 89.2K. We make the taxonomy, not the ranker, the object of study: an error analysis shows that 98 of 101 failures were cases where the request never named the expected capability, and repairing the taxonomy on one half of the benchmark lifts accuracy on the held-out half from 0.556 to 0.742, and over the whole benchmark from 0.661 to 0.849 ($p = 3.2e-15$). We also report three negative results: the outcome ledger does not measurably improve routing at the scale of our evidence, BM25 ties the full router on exact-skill selection, and per-invocation token cost rises as accuracy rises.

KEYWORDS agent skills; tool selection; capability routing; context engineering; token efficiency; taxonomy design

Introduction

An agent skill is a small, self-contained instruction bundle: a markdown file that tells the agent how to perform one kind of job, optionally with reference files, scripts and assets. Skills are the unit of reuse in modern coding agents. They are also cheap to write and expensive to carry. A machine that installs skills from marketplaces, from plugins and from the user's own collection accumulates them faster than any agent can read them.

The obvious answer, and the one most agent hosts take, is to expose every skill to the model and let it choose. This works until it does not. The catalog itself becomes a fixed cost on every single request. On the machine studied here, 1,387 skills were visible to the agent, and the skills that the agent actually needed were a tiny minority of them.

We built a capability-routing system to fix this, and we report it with an evaluation rather than an anecdote. The contributions are:

A measurement of the problem. We quantify the cost of a large catalog and show that the useful subset is small.

A taxonomy design that treats capability tags as claims that need evidence. We define four evidence channels with different strengths and we make the auditor and the tagger agree on what counts as evidence. Before the repair, 845 of 1,318 local and plugin skills were unreachable by construction.

A router that resolves a need to exactly one skill. The router's design follows a simple principle: the cheapest skill that fully covers the need wins. Getting there required replacing three naive mechanisms, each of which we document because they are common.

An evidence-driven learning loop that scores skills from what a session actually did with them. A skill is good when the files it produced survived the session, and bad when they were deleted.

An error analysis that says where the remaining failures live, and a repair that follows from it. Categorising the failures by whether the request named the expected capability at all showed that 98 of 101 failures were inference misses and only three were ranking misses. Repairing the taxonomy on one half of the benchmark lifted accuracy on the held-out half from 0.556 to 0.742, and over the whole benchmark from 0.661 to 0.849.

Three negative results, stated plainly. The learning loop does not measurably improve routing at our scale; a textbook BM25 retriever ties the full router on exact-skill selection; and routing accuracy and per-invocation token cost move in opposite directions.

The rest of the paper describes the system, the evaluation methodology, and the results, and closes with the limits of what we measured.

Background

Agent hosts differ in how they surface skills, but the common shape is a registry. Each installed skill contributes an entry with a name, a description and, often, the full body of the skill. The catalog is either injected into the system prompt or offered as a searchable tool. Both forms pay for the whole catalog, and both make the model's task of selection harder as

the catalog grows.

Three properties of skill catalogs make the problem different from ordinary retrieval. First, the catalog is mostly irrelevant to any given request: the useful skills for a code review are a small, stable subset. Second, description text is short and highly lexical, so a bag-of-words matcher is a surprisingly strong baseline and a surprisingly hard thing to beat. Third, the cost that matters is not the cost of the right skill, which is paid once, but the cost of the catalog, which is paid on every request.

This suggests a division of labour. The catalog problem is a gating problem: remove what is not needed before the model ever sees it. The selection problem is a ranking problem: given what remains, choose one skill, and choose a cheap one.

Related Work

Tool selection. Much of the prior work on tool choice is about learning when to call a tool rather than which of thousands to call. Toolformer learns which API calls to emit; Gorilla and ToolLLM evaluate and train models for API selection; ReAct and Reflexion structure the loop in which a selection is made and then judged by what follows. Our setting differs in two respects. The candidate space is instruction bundles rather than API functions, and the selection is made by a deterministic program rather than by the model, so the router cannot be trained by gradient descent. We borrow Reflexion's central intuition, that the quality of a trajectory is observable after the fact, and apply it to skill selection instead of to reasoning steps.

Retrieval and lexical matching. Our strongest baseline is a lexical matcher in the style of BM25 over skill descriptions, and it is competitive. We keep it in the evaluation rather than hiding it, because the honest finding is that a large part of routing is lexical matching and the router's contribution is concentrated in a smaller number of cases.

Context efficiency. A line of work argues that long contexts degrade model performance, not just cost: the Lost in the Middle study shows that relevant information in the middle of a long context is used less reliably, RULER measures effective context length against nominal length, and recent work on context rot documents degradation with irrelevant material. The Chroma context-rot study is the clearest statement of the practical version of this: a catalog of mostly irrelevant entries is worse than a small one, not merely more expensive. Our catalog gate is an engineering response to that finding.

Memory and context management. Letta and MemGPT treat conversation state as something to be actively managed rather than passively accumulated, and the token-guard and compaction tools that many agent hosts ship are practical versions of the same idea. The learning loop in this paper sits in this family: it decides what is worth keeping, but it keeps preferences about capabilities rather than facts about tasks.

Capability taxonomies. The router needs a vocabulary of job kinds. We build it explicitly, in the style of WordNet-style lexical resources, and we treat it as a maintained artefact with an auditor rather than as a one-off annotation pass. Section 4.2 explains why that distinction mattered more than expected.

The System

The system has four coupled parts. The catalog gate and the parking mechanism decide what the model can see. The taxonomy and the registry decide what each skill is good for. The router turns a free-text need into one skill and loads it. The learning loop turns what happened in past sessions into scores that feed back into the router. A fifth part, the native skill-evolution channel, lets a broken or missing skill be reported from the command line; it was present in the host but unreachable, and repairing it was a precondition for the rest of the work.

Catalog gating and parking

The host reads three external skill roots plus its own data directory, and exposes all of them by default. A single configuration key disables the three external roots, which removes 1,343 of the 1,387 visible skills without touching the data directory root, which the host always scans.

Skills that live in the always-scanned root cannot be gated by configuration. They are parked instead: the directory is moved out of the scanned root into a holding directory, with a manifest recording what was moved so that any single skill can be restored. Parking is a move, not a copy, so it costs no disk space and is reversible by moving the directory back.

The residual 44 skills are the token-control skills, the router itself, the forge family and the native evolution channel. The catalog listing cost falls from an estimated 1.92M tokens to 89.2K, a 95.4% reduction, and the visible catalog falls from 1,387 entries to 44, a 96.8% reduction.

The capability taxonomy

A capability tag is a claim that a skill provides a particular kind of job. We treat every tag as a claim that needs evidence, and we define four evidence channels of decreasing strength.

Files. A skill that ships reference/critique.md critiques. Filename stems are matched exactly, never as substrings, so that a skill shipping type.md does not match every TypeScript skill.

Strong keywords. A multi-word phrase in the description, such as write documentation or unit test. A phrase is strong evidence because it is specific.

Weak keywords. A single word in the skill's name, such as browser or review. Weak evidence, and the source of most tagging errors.

Exact and required keywords. Some names must match a whole token (cli must not match clickmeeting) and some capabilities only apply to skills whose name contains a given string (the automation capability only applies to skills named ... automation).

Two properties of this design are worth stating because they were learned the hard way. First, matching must be done per name token, not by raw substring. Substring matching made form match transform, test match latest, git match digit and lock match block, and every one of those errors produced a wrong skill at routing time. Second, a single common noun used as a strong keyword is not strong evidence: process alone made one

capability cover 159 of 1,318 skills, which is no coverage at all.

The pre-fix system had a subtler defect. The auditor that reported stale tags and the tagger that produced them used different definitions of evidence, so the auditor called correct tags stale. The auditor now uses the same rules as the tagger, which is what makes its output actionable rather than merely alarming.

The router

The router takes a free-text need and returns exactly one skill, together with the reason it was chosen. It resolves and then acts: the chosen skill's body is loaded, not merely reported. Scoring proceeds in five stages.

Capability inference. The request is matched against every capability's keyword lists. A strong phrase counts double a weak name word. This matters because a request typically names its object strongly and its context weakly: write unit tests for the auth module is a request for testing, and auth is what the tests are about. Under flat inference an auth-flavoured skill outranked the testing skill.

Head capability. Among the capabilities the request names, the one mentioned first carries the intent. Generate an illustration for the landing page wants an illustration, on a landing page. When two capabilities are named at the same position, the more specific one wins, measured by how many skills carry it.

Capability match. A skill scores by the capabilities it carries that the request also names. Primary capabilities dominate; a capability named only in passing contributes little.

Name relevance and ledger. The score of a skill whose name shares a word with the request, and the skill's learned quality score, both act as tie-breakers. A negative ledger score is applied as a capped penalty rather than a ban, so a rejected skill stays reachable as a last resort.

Cost gate. The final selection prefers the cheapest skill that covers the need. The naive form of this rule, comparing a candidate against the cheapest cover in the list, deleted the better match: a 3.5K-token skill that matched the request exactly died because a 64-token skill that matched it loosely was cheaper. The rule we ship is Pareto: a skill is too expensive only if some other skill matches at least as well and costs a fraction of it. A hard budget is also available as an option, and we report its effect in Section 7.4.

The learning loop

The router is told which skills worked by watching what happens in a session. A plugin hook fires on every user prompt and after every tool use. When a skill is loaded, the hook remembers it. When the session then edits or writes a file, that is an accept: the skill's output was good enough to build on. When the session then deletes a file that the same skill had produced, that is a reject.

The rule is deliberately conservative. A delete that does not target a file the skill produced is not evidence against that skill, and neither is a file the skill never touched. An early version counted any delete in the window as a rejection, which produced systematically pessimistic scores. The evidence rules are also enforced in a batch learner that replays stored transcripts, which is how the corpus of 304 sessions was scored and how 19 skills acquired a quality score.

The hook is a failure-tolerant boundary. Every path exits zero. A hook that has no opinion must never block the agent.

The native evolution channel

Hosts expose a mechanism for reporting that a skill is missing a step or uses a wrong default, and for turning such reports into improved skills. On this machine the mechanism existed but could not be reached: its calls are not top-level JSON-RPC methods but live on a separate service contract, reachable only through a transport method that the command line did not speak. The repair was a small client that implements the transport, plus the instruction bundle the runtime asked for at startup and which had never existed. Signals and proposals can now be reported, listed and cancelled from the shell, verified end to end against a live host.

Two operations, reverting and running the lifecycle, remain unavailable: the embedded runtime answers them with a hard error. We report this as a limit of the environment rather than of the design.

Evaluation

We ask four questions. RQ1: how well does the router select a skill, and how does it compare with weaker selection strategies? RQ2: how much of the gain comes from each component? RQ3: what does the system cost in tokens, and what is the shape of the accuracy-cost frontier? RQ4: what happens to the catalog and the taxonomy?

Benchmark

We constructed 242 requests labelled by the capability they should resolve to, covering 61 of the 78 capabilities, and authored 51 further requests together with the single unambiguous skill they should resolve to. Of the 293 authored requests, 58 name exactly one skill and therefore form the strict subset. A further 5 cases were extracted from stored sessions, giving 298 cases in total and 63 strict cases.

Two properties protect the benchmark from circularity. The requests were written in the language a person would type, and they deliberately avoid the taxonomy's exact keyword strings where a natural paraphrase exists; the harness computes a lexically isolated subset, defined as the cases whose request shares no exact keyword string with the expected capability's lists, and reports accuracy on that subset separately. And the strict subset names one skill rather than a category, so it measures selection rather than classification.

The benchmark is split in half by a stable hash of the case id. Every taxonomy change reported in this paper was made using the dev half alone, so the test half is a held-out measurement of the repair. We say plainly that this is not a clean held-out evaluation of the system as a whole, because the router's rules and the taxonomy's structure were both designed while every case was visible. It is a held-out measurement of one specific intervention.

We attempted the obvious alternative: using the user's own history as ground truth. Across 304 transcripts holding 757 user messages there were 67 skill invocations, and only 5 request-to-skill pairs could be associated with any confidence, because skill loads are preceded by long tool sequences. That scarcity is itself a finding: usage-based ground truth is rare, which is why the system learns from outcomes rather than from request-to-skill pairs, and why a hand-labelled benchmark is needed.

Systems compared

We compare eight systems. Random samples uniformly from the catalogue. Name-only ranks by name-word overlap. BM25 is Okapi BM25 over skill name and description with no taxonomy, no ledger and no cost gate: a real retrieval baseline, and the one a reviewer is right to ask for. Keyword-substring is a faithful reimplementation of the pre-fix router: flat strong-plus-weak inference by raw substring, substring name and description hits, additive capability overlap, and a cost gate comparing against the cheapest cover. No-ledger is the shipped router with the outcome ledger neutralised. No-cost-gate is the shipped router without the Pareto gate. Full-budget is the shipped router restricted to skills costing at most 2,000 tokens whenever such a skill covers the need. Full is the shipped router.

Metrics

Capability top-1 is the share of cases where the chosen skill carries the expected capability. Capability top-3 and MRR extend this to a ranked list. Strict top-1 is the share of strict-subset cases where the chosen skill is exactly the expected one. On-topic rate is the share of cases where at least a fifth of the request's content words appear in the chosen skill's name or description; it consults no taxonomy at all, so it is comparable across every system including random. Cost is the on-invoke token cost of the chosen skill, as mean and median.

Threats to Validity

The benchmark is small and written by one author, so its distribution of requests reflects one person's habits, and its labels reflect one person's reading of what each request means. Capability labels are the weaker of the two labelling schemes: a skill can carry the expected capability and still be a poor answer, which is why we report the strict subset and the taxonomy-independent on-topic rate alongside it. The evaluation runs against a registry of 4,225 skills on one machine with one catalogue of third-party skills, so the results should not be read as generalising to other collections. The learning loop is evaluated with only 19 scored skills, which is too few to measure its effect, and we report that as a null result rather than a positive one.

Results

Table 1 gives the headline numbers over all 298 cases, of which 63 carry a single expected skill. The remaining tables give the significance of the differences, the intervals around the estimates, the lexically isolated subset, the accuracy-cost frontier, the error analysis and the system-level effects.

Routing accuracy

The shipped router reaches 0.849 capability top-1 accuracy against 0.614 for the pre-fix matcher, a gain of 23.5 points, and 0.473 for Okapi BM25 over skill descriptions. Mean reciprocal rank rises from 0.678 to 0.878, and from 0.570 for BM25. The strict subset tells a different story: the shipped router scores 0.397 against 0.206 for the pre-fix matcher and 0.397 for BM25.

That tie is the most important number in the paper. When the question is which capability a request belongs to, the taxonomy decisively beats lexical retrieval. When the question is which exact skill to load, a textbook BM25 retriever over skill descriptions matches the full router, and the pre-fix matcher is far behind. The taxonomy buys capability coverage, not skill disambiguation.

Significance

The comparisons are paired, case by case. Against the pre-fix matcher the shipped router is right where the baseline is wrong on 80 cases and wrong where it is right on 10, which McNemar's exact test puts at a raw p of 1.1e-14 and a Holm-corrected p of 4.2e-14 across the seven baselines. The paired bootstrap puts the MRR difference at +0.200 with a 95 percent interval of [0.155, 0.247].

Against BM25 the router is right where BM25 is wrong on 119 cases and wrong where it is right on 7, an MRR difference of +0.308 with an interval of [0.259, 0.356] and a corrected p of 2.0e-26. The capability-level metric separates the two systems decisively; the strict metric does not, and we say which is which.

Cases are not independent: several cases expect the same capability, and the router behaves consistently on them. Resampling capabilities rather than cases widens the interval, and we report both. The conclusion does not depend on which is used.

On-topic rate

The taxonomy-independent on-topic rate is where the router's advantage over the pre-fix matcher is largest: 0.799 against 0.460, and 0.560 for BM25. The pre-fix matcher frequently selects a cheap skill that shares a word with the request but has nothing to do with the job. The shipped router selects a skill whose description is actually about the request.

Ablations

Neither ablation changes capability top-1, which stays at 0.849 in all three configurations. Removing the outcome ledger leaves strict top-1 at 0.413 against 0.397 for the shipped router, so at this scale the ledger is indistinguishable from noise. Removing the Pareto cost gate leaves both metrics unchanged and cuts mean token cost from 3,114 to 2,881. The gate is cheap and safe but close to inert.

The ledger result is a null result and we report it as one. With only 19 skills carrying a score, and most benchmark cases not involving them, the ledger cannot move the aggregate. We believe the mechanism is sound but that its value is in long-tail cases rather than in the mean, and we have not measured that.

The taxonomy ablation

The paper claims the taxonomy is the load-bearing part, so we measured it instead of asserting it. The same router, the same

benchmark and the same cost gate were run against the pre-repair keyword lists, with the registry rebuilt from scratch so that the skills are tagged the way they would have been tagged before. Capability top-1 moves from 0.661 to 0.849, and mean reciprocal rank by 0.164 with a 95 percent paired bootstrap interval of [0.126, 0.204]. The repaired taxonomy is right where the original is wrong on 58 cases and wrong where it is right on 2, a paired difference that McNemar's exact test puts at p = 3.2e-15. Nothing else changed.

The repair was guided by failures on one half of the benchmark, so that half is no longer an unbiased estimate: it scores 0.959. The other half was never looked at during the repair and scores 0.742, against 0.556 for the pre-fix matcher on the same cases. The gap between the halves, 21 points, is the size of the optimism we cannot rule out elsewhere, and we would rather report it than hide it. On the held-out half the difference is still significant: 36 cases favour the repaired taxonomy, 8 favour the original, p = 2.5e-05.

The accuracy-cost frontier

Accuracy and per-invocation cost move in opposite directions. The pre-fix matcher spends a mean of 592 tokens per invocation; the shipped router spends 2,881, nearly five times more, while improving accuracy. The reason is visible in the wins: on the cases the shipped router gets right and the pre-fix matcher gets wrong, the mean cost of the chosen skill rises from 249 tokens for the pre-fix matcher to 2,727 for the shipped router. The pre-fix matcher was cheap because it was often wrong, and being wrong is cheap.

A hard budget recovers most of the cost. Restricting the router to skills of at most 1,000 tokens whenever such a skill covers the need gives a mean of 1,149 tokens, a 60 percent reduction, for 2.3 points of capability top-1 and 14.3 points of strict top-1. The frontier is therefore shallow: cost is cheap to recover, accuracy is not.

Catalog and taxonomy

The catalog effects are unambiguous. Skills visible to the agent fall from 1,387 to 44, the catalog listing cost falls from an estimated 1.92M tokens to 89.2K, the number of capabilities rises from 51 to 78, the number of skills that no capability could reach falls from 845 to 0, and the number of capabilities with no provider stays at 0.

The registry itself grew during this work, from 4,098 to over 4,200 skills, because the builder was taught to read the host's built-in skill directory and its plugin directory. Before that change, 14 installed skills and a bundle of plugin skills were invisible to the router: they could be invoked by name but never selected. Making the router's view match the host's view was necessary before any evaluation could be trusted.

Lexically isolated cases

A case is lexically isolated when its request shares no exact keyword string with the expected capability's keyword lists. Which cases qualify depends on the lists used to define the subset, so the harness measures both: the shipped taxonomy's lists, and the pre-repair lists, which are a fixed reference independent of the current system. Every system is scored on the same subset.

Under the shipped taxonomy's lists, 66 cases are isolated and the shipped router scores 0.364 against 0.273 for the pre-fix matcher and 0.273 for BM25. Under the pre-repair lists, 126 cases are isolated and the shipped router scores 0.659 against 0.444 and 0.349. The router is ahead on both definitions.

The absolute numbers are the interesting part. Isolated cases are hard for every system: the shipped router's 0.364 on the 66-case subset is less than half its 0.849 over the whole benchmark. A request that avoids the taxonomy's vocabulary is a request the router cannot route on evidence, and it says so.

Error analysis

Of the 45 capability-level failures, 42 are inference misses: the request never named the expected capability, so no skill carrying it could win. Two are ranking errors and one is a near miss where the right capability was in the top three. On the strict subset the pattern is different: 27 of 38 failures choose a sibling skill that carries the right capability, and only 11 choose the wrong capability.

That is the argument for spending effort on the taxonomy rather than on the ranker, and it is why the taxonomy ablation above matters more than any ranking change. It also explains the remaining ceiling: the strict failures are a disambiguation problem, and disambiguation needs either a better model of skill intent or a human in the loop.

Robustness

Deleting the strongest matching keyword from a request and routing it again is brutal: the expected capability is still inferred in only 18.5 percent of cases, and the same skill is chosen in 34.1 percent. The router is keyword-driven, not semantic, and it says so.

We report this because it bounds the claim. The system does not understand the request; it recognises evidence in it. A request that says the same thing in different words is a different request to the router, and a learned or model-based router would be the obvious next step.

Table 2. Routing accuracy over all 298 cases. Strict top-1 is measured on the 63 cases with one unambiguous skill.

system	cap top-1	cap top-3	MRR	strict top-1	on-topic	mean tok	median tok
random	0.010	0.040	0.022	0.000	0.050	2,153	1,294
name-only	0.198	0.332	0.292	0.127	0.248	703	336
keyword-substring (pre-fix)	0.614	0.708	0.678	0.206	0.460	592	214
no-ledger	0.849	0.889	0.876	0.413	0.802	2,801	1,539
no-cost-gate	0.849	0.893	0.878	0.397	0.795	3,114	1,683
full-budget (2K)	0.829	0.859	0.847	0.365	0.711	1,402	1,011
full (shipped)	0.849	0.893	0.878	0.397	0.799	2,881	1,550

Table 3. Paired significance of the shipped router against every baseline. McNemar's exact test on capability top-1 over the discordant pairs; a paired bootstrap gives the interval for the MRR difference; p-values are Holm-corrected across the six baselines.

baseline	full only	base only	MRR diff	CI (cases)	CI (clust)	p	p (Holm)
random	250	0	+0.856	[0.820, 0.891]	1.1e-75	7.7e-75	
name_only	198	4	+0.585	[0.536, 0.633]	2.1e-53	1.3e-52	
bm25	119	7	+0.308	[0.259, 0.356]	2.1e-27	1.1e-26	
keyword_substring	80	10	+0.200	[0.155, 0.247]	1.1e-14	4.2e-14	
full_budget	8	2	+0.030	[0.013, 0.050]	0.11	0.33	
no_ledger	0	0	+0.002	[-0.001, 0.004]	1	1	
no_cost_gate	0	0	+0.000	[-0.001, 0.001]	1	1	

Table 4. Wilson score intervals on the headline numbers.

quantity	n	point estimate	95% CI (Wilson)
full cap top1	298	0.849	[0.804, 0.885]
baseline cap top1	298	0.614	[0.558, 0.668]
full on topic	298	0.799	[0.749, 0.840]
baseline on topic	298	0.460	[0.404, 0.516]
full strict top1	63	0.397	[0.285, 0.520]
baseline strict top1	63	0.206	[0.125, 0.322]

Table 5. A split of the cases the design was not tuned on, by a stable hash of the case id.

split	n	full top-1	baseline top-1	full strict	baseline strict
dev	147	0.959	0.673	0.387	0.226
test	151	0.742	0.556	0.406	0.188

Table 6. Accuracy on the lexically isolated subset, where the request shares no exact keyword string with the expected capability.

system	shipped lists (n=66) top-1	shipped lists (n=66) top-3	pre-repair lists (n=126) top-1	pre-repair lists (n=126) top-3
random	0.015	0.008	0.040	
name-only	0.182	0.159	0.254	
keyword-substring (pre-fix)	0.273	0.444	0.540	
no-ledger	0.364	0.659	0.754	
no-cost-gate	0.364	0.659	0.762	
full-budget (2K)	0.288	0.627	0.698	
full (shipped)	0.364	0.659	0.762	

Table 7. The accuracy-cost frontier: the same router under a hard token budget.

token budget	cap top-1	strict top-1	mean cost	median cost
500	0.805	0.206	1,396	360
1000	0.826	0.254	1,149	734
2000	0.829	0.365	1,402	1,011
4000	0.846	0.397	1,891	1,391
8000	0.849	0.381	2,268	1,518
none	0.849	0.397	2,881	1,550

Table 8. Every capability-level failure of the shipped router, split by where the routing went wrong.

failure category	count
inference error: the request never named the capability	42
ranking error: right capability was inferred but not ranked	2
ranking error: right capability was in the top three	1
strict: right capability, sibling skill chosen	27
strict: wrong capability	11

Table 9. Taxonomy ablation: the shipped router, the same benchmark and the same cost gate, with the taxonomy's keyword lists replaced by the pre-repair set.

taxonomy	cap top-1	strict top-1	MRR	on-topic	mean cost
v1, before the repair	0.661	0.381	0.714	0.805	2,587
v2, after the repair	0.849	0.397	0.878	0.799	2,881

Table 10. System-level effects of the catalog gate and the taxonomy work.

quantity	before	after
skills visible to the agent	1,387	44
catalog listing cost (tokens, est.)	1,922,567	89,205
capabilities in the taxonomy	51	78
skills no capability could reach	845	0
tags unsupported by their evidence	78	0
skills in the registry	4,098	4,225

Limitations

The evaluation is one machine, one catalogue of third-party skills and one author. The benchmark contains 293 authored requests and 63 strict cases; the strict subset, which is the metric we trust most, is small.

Capability labels are coarse. A skill can carry the expected capability and still be a poor answer. There is no human-preference study, and that would be the natural next step.

The audit numbers are constructive rather than causal. We repaired the auditor so that it uses the same evidence rules as the tagger, and the count of unsupported tags went to zero because the definition changed, not because 78 bad tags were individually found and fixed. The honest statement is that the two tools now agree, and that they did not before.

The taxonomy repair was guided by failures on the dev half, so the dev half is no longer an unbiased estimate of accuracy. The test half is, and we report it, but the gap between them is the size of the optimism we cannot rule out elsewhere.

The learning loop is evaluated with 19 scored skills. Its null result is a statement about the amount of evidence available, not proof that the mechanism cannot work.

Two native evolution operations, reverting a skill and running its lifecycle, are unavailable in the embedded runtime and could not be evaluated.

The catalog gate is host-specific. It relies on a configuration key whose name and semantics are properties of this host, not a standard.

Conclusion

Routing a large agent skill library is mostly a gating problem and a ranking problem, and treating it as either one alone leaves value on the table. Gating alone cuts the visible catalogue by 97%. Ranking alone, with a careful taxonomy, lifts capability accuracy from 0.614 to 0.849 against a faithful reconstruction of the pre-fix matcher.

The load-bearing part is the taxonomy, and we now have the measurement to say so rather than the assertion. With the router, the benchmark and the cost gate held fixed, replacing the taxonomy's keyword lists with a repaired set moves accuracy from 0.661 to 0.849. The error analysis that motivated the repair says the same thing from the other side: before it, 98 of 101 failures were requests that never named the expected capability, and only three were ranking errors.

The most useful negative results are that the outcome ledger does not yet earn its place in the router, that a textbook BM25 retriever ties the full router when the question is which exact skill to load, and that accuracy and token cost pull against each other. Each is a statement about the evidence available at this scale rather than about the design, and all three point at the same next step: a larger, human-judged evaluation.

References

[1] Shick, J. et al.. *Toolformer: Language models can teach themselves to use tools*. Advances in Neural Information Processing Systems 36, 2023..

[2] Patil, S. G. et al.. *Gorilla: Large language model connected with massive APIs*. arXiv:2305.15334, 2023..

[3] Yao, S. et al.. *ReAct: Synergizing reasoning and acting in language models*. International Conference on Learning Representations, 2023..

[4] Shinn, N. et al.. *Reflexion: Language agents with verbal reinforcement learning*. Advances in Neural Information Processing Systems 36, 2023..

[5] Qin, Y. et al.. *ToolLLM: Facilitating large language models to master 16,000+ real-world APIs*. International Conference on Learning Representations, 2024..

[6] Liu, X. et al.. *AgentBench: Evaluating LLMs as agents*. International Conference on Learning Representations, 2024..

[7] Jimenez, C. E. et al.. *SWE-bench: Can language models resolve real-world GitHub issues?*. International Conference on Learning Representations, 2024..

[8] Robertson, S. and Zaragoza, H.. *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval 3(4), 2009..

[9] Lewis, P. et al.. *Retrieval-augmented generation for knowledge-intensive NLP tasks*. Advances in Neural Information Processing Systems 33, 2020..

[10] Liu, N. F. et al.. *Lost in the Middle: How language models use long contexts*. Transactions of the Association for Computational Linguistics 11, 2023..

[11] Hsieh, C.-Y. et al.. *RULER: What's the real context size of your long-context LLMs?*. arXiv:2404.06654, 2024..

[12] Packer, C. et al.. *MemGPT: Towards LLMs as operating systems*. arXiv:2310.08560, 2023..

[13] Chroma Research. *Context rot: How increasing input tokens impacts LLM performance*. Technical report, 2025..

[14] Anthropic. *Agent Skills specification and context engineering guidance*. Documentation, 2025..

[15] D. Osman and contributors. *superpowers: a software development methodology for coding agents*. Open-source project, obra/superpowers..

[16] Miller, G. A.. *WordNet: A lexical database for English*. Communications of the ACM 38(11), 1995..